

P1302R1

Implicit Module Partition Lookup

Published Proposal, 2019-01-21

This version:

<https://wg21.link/p1302r1>

Authors:

[Isabella Muerte](#)

[Richard Smith](#)

Audience:

EWG

Project:

ISO/IEC JTC1/SC22/WG21 14882: Programming Language — C++

Current Render:

[P1302](#)

Current Source:

slurps-mad-rips/papers/proposals/implicit-module-partition.bs

Abstract

We seek to enforce a particular form of module partition lookup as a "fast path" opportunity for building modules, with minimal impact to existing code, while allowing for incremental modularization of user's code bases. This is not a convention, but a requirement for conformance for all vendors whose code will run on an abstract machine

Table of Contents

1 Revision History

1.1 Revision 1

1.2 Revision 0

2 Motivation

3 Design

4 Examples

4.1 Hierarchical

4.2 Splayed

5 Wording

§ 1. Revision History

§ 1.1. Revision 1

Added actionable language for inclusion into the standard. We now state that splayed layouts are the minimum requirement and any hierarchy or module name enforcement is solely the responsibility of the build system.

§ 1.2. Revision 0

Initial Release 🎉

§ 2. Motivation

As the advent of modules approaches, build systems and developers are still waiting on the final bits that will make it into C++20. However, because of various limitations, the standard cannot *enforce* specific convention, merely encourage them. This paper seeks to provide a possible convention that reduces work required by build systems, general effort from compilers (e.g., they will not need to implement a local socket based server for information passing), and to make the lives of developers easier as they will get to experience a fairly consistent development process when moving between or across projects.

§ 3. Design

The design for this behavior is as follows:

A so-called *module entrypoint* is placed inside of a directory. A *module entrypoint*, as defined in [§5 Wording](#) is a file. The name of this *entrypoint* is user defined, with a fallback to a file with the base name `module` and possibly some file extension. We do not enforce the requirement of a file extension so that operating systems without them can do as they please. This also allows the community to eventually select a "better" file extension. That said, the author recommends `cxx` as that is the environment variable used by plenty of build systems to represent the C++ compiler, where as `CPP` represents the **C PreProcessor**. Additionally, moving libraries from non-module APIs to modularized APIs can have a path of least resistance instead of requiring something that has nothing to do with the name C++, like `cppm` or `mxx`.

Therefore, if a module's interface has `import :list`, the compiler will look for a file with a base name of `list`, and (optionally) the same file extension as `module`.

This behavior only occurs during the construction of a BMI. If dependent modules are not yet compiled, or if users are expecting an object file to magically pop out of the compiler, they will be surprised when the compiler gives an error.

The building of object files is still left up to build systems so that existing distributed build system workflows are not interrupted (such as in the case of tools like Icecream, distcc, or sccache).

Where things get interesting is when a user desires to import *another module* into a *module entrypoint*. Given the following directory layout:

```
├── src
│   └── core
│       ├── module.cxx
│       ├── list.cxx
│       └── io
│           ├── module.cxx
│           └── file.cxx
```

We can assume that, perhaps, the source for `core/module.cxx` looks something like:

```
export module name;

export import core.io;
import :list;
```

In other languages, this would imply that the compiler has to now recurse into the `core/io` directory. **We do not do this.** Instead, the build system is required to have seen the `export import` and passed that directory along to the compiler first. This is a combination of the various modules systems, but has the following properties:

1. It does not make the compiler a build system
2. Existing work that has been done to handle dependency management does not need to be thrown away.
3. The `core.io` module *does not* have to exist in the directory `core/io`. Rather, it can exist technically anywhere.
4. Build systems are free to enforce the name of a module to its location on disk, while also permitting others to ignore it entirely.
5. Build systems can have a guaranteed fallback location if developers don't want to have to manually specify the location of each and every module.
6. This doesn't actually tie the compiler to a filesystem approach, as this is just a general convention.
7. Build systems are free to implement, additional conventions, such as the Pitchfork or Coven filesystem layout and enforce it for modules having legacy non-module code in the same project layout.
8. It allows developers to view modules as hierarchical, even if they aren't. This means that, if treating modules as a hierarchy becomes widespread enough, the standard could possibly enforce modules as hierarchies in the future.
9. Platforms where launching processes are expensive can take advantage of improved throughput when reading from files.
10. Build systems and compilers are free to take an optimization where only the modified times of a directory are checked before the contents of each directory are checked. On every operating system (yes, every operating system), directories change their timestamp if any of the *files* contained within change, but do not update if child directories do as well. While some operating systems permit mounting drives and locations without modified times, doing so breaks nearly every other build system in existence. Thus we can safely assume that a build system does not need to reparse or rebuild a module if its containing directory has not changed.

§ 4. Examples

The following two examples show how implicit module partition lookup can be used for both hierarchical and "splayed" directory layouts.

§ 4.1. Hierarchical

This sample borrows from the above example. Effectively, to import `core.io`, one must build it before building `core` simply because the build system assumes that `core.io` refers to a directory named `core/io` from the project root.

```
├── src
│   ├── core
│   │   ├── module.cxx
│   │   ├── list.cxx
│   │   └── io
│   │       ├── module.cxx
│   │       └── file.cxx
```

This behavior is *not* enforced by the compiler, but rather by the build system. If a build system does not support a *hierarchical* implicit lookup, it can at least support a *splayed* implicit lookup

§ 4.2. Splayed

This approach is one that might be more commonly seen as C++ developers move from headers to modules.

```
├── core
│   ├── module.cxx
│   └── list.cxx
└── io
    ├── module.cxx
    └── file.cxx
```

In the above layout, `core.io` is located in `./io`, rather than under the `./core` directory. A sufficiently simple build system could be told that `core.io` resides under `./io` and not to rely on

some kind of hierarchical directory layout.

§ 5. Wording

The following is to be placed into the current working draft at a location within close proximity to, or adjacent to, the current merged modules wording:

General

¹ This subclause describes operations to be supported regarding modules, their *implicit* partitions, and interactions within the filesystem.

² A *module container* is a collection of files represented by a *directory*. [fs.general]

³ A *module entrypoint* is a filesystem object within a *module container* that holds the purview of an *exported module*.

⁴ An *implicit module partition* is a filesystem object adjacent to a *module entrypoint*.

Conformance

¹ Conformance is required for all vendors whose implementations create a final program to be run on the abstract machine

² Compilers must provide a flag for users to declare a *module container* as a filesystem directory.

³ Compilers must provide a flag for users to declare the name of a *module entrypoint*

⁴ Compilers must use a predefined *module entrypoint* if none is provided by a user. This entrypoint must be a file with a base name of *module*. The extension of the file is currently left unspecified, but should be one of the several file extensions that have been historically used. (e.g., `.c++`, `.cpp`, `.cxx`, and `.cc`, et. al.)

Behavior

¹ In the *module entrypoint's preamble*, when an implementation encounters a *module partition* import, it needs to look for an *implicit module partition* with the same base name as the *module partition*